Web Hosting & Cloud Technologies

Case Study Compendium

12 Real-World Scenarios
Covering Hosting Architectures, Security, Scalability & More

Case Study 1

Shared Hosting vs VPS vs Cloud Hosting

Scenario	A small business website is facing slow performance as traffic increases. The business owner needs to evaluate different hosting options to find the most appropriate solution.

Activity: Comparative Analysis & Recommendation

1.1 Overview of Hosting Types

The three major categories of web hosting each offer distinct trade-offs in terms of control, cost, performance, and scalability. Understanding these differences is essential for making an informed hosting decision.

Factor	Shared Hosting	VPS Hosting	Cloud Hosting
Cost	Very Low ($2–$10/mo)	Moderate ($20–$80/mo)	Variable (pay-as-you-go)
Performance	Low (shared resources)	Medium (dedicated slice)	High (on-demand resources)
Scalability	Very limited	Manual scaling	Auto-scaling
Control	Minimal	Root access	Full control via console
Uptime SLA	~99.5%	~99.9%	~99.99%
Best For	Blogs, small sites	Growing businesses	High-traffic, enterprises

1.2 Detailed Analysis

Shared Hosting: Multiple websites share a single server's CPU, RAM, and storage. It is the most affordable but suffers from the 'noisy neighbour' problem — traffic spikes on other sites can degrade your performance. Suitable only for low-traffic static websites.

VPS (Virtual Private Server): A physical server is partitioned into isolated virtual machines. Each VPS has guaranteed resources. It offers greater control and performance than shared hosting but requires technical expertise to manage. Scaling is manual and limited by the physical host.

Cloud Hosting: Resources are distributed across multiple servers in a cloud infrastructure (e.g., AWS, Azure, GCP). It offers elastic auto-scaling, high availability, and a pay-per-use model. It is the most resilient and scalable option.

1.3 Recommendation

Verdict	Cloud Hosting is recommended for this small business. Although the initial cost is higher than shared hosting, the auto-scaling capability ensures performance is maintained as traffic grows. A starter plan on a managed cloud platform (e.g., AWS Lightsail or DigitalOcean App Platform) can bridge cost concerns while providing superior performance.

1.4 Migration Roadmap

- Step 1: Begin with shared hosting to minimise initial cost
- Step 2: Monitor traffic growth over 3–6 months
- Step 3: Migrate to VPS when monthly visitors exceed 10,000
- Step 4: Upgrade to cloud hosting when concurrent users exceed 500
- Step 5: Implement CDN alongside cloud hosting for global reach

Case Study 2

E-Commerce Website During Peak Traffic

Scenario	An online store faces heavy traffic during a festival sale and risks downtime. The platform must handle sudden 10x traffic spikes without service disruption.

Activity: High-Availability Architecture Design

2.1 Problem Analysis

Festival sales (e.g., Diwali, Black Friday) cause unpredictable traffic spikes. Without proper architecture, even a 2x spike can cause database overload, page timeouts, and complete downtime — resulting in lost revenue and customer trust damage.

2.2 Proposed Architecture

Architecture	Multi-tier cloud architecture with auto-scaling, load balancing, CDN, and database read replicas deployed across multiple availability zones.

Layer	Component	Purpose
Edge	CloudFront / Cloudflare CDN	Cache static assets globally
Traffic	Application Load Balancer	Distribute requests across servers
Compute	Auto Scaling Group (EC2/GCE)	Add/remove instances automatically
Application	Containerised App (Docker/K8s)	Stateless, horizontally scalable
Cache	Redis / ElastiCache	Session data and frequent queries
Database	RDS with Read Replicas	Handle high read load
Storage	S3 / Cloud Storage	Product images, assets

2.3 Scaling Strategy

- Pre-scaling: Provision 2x capacity 24 hours before the sale starts
- Auto-scaling: Configure CPU threshold at 60% to trigger new instance launch
- Database: Enable connection pooling (PgBouncer) and add 3 read replicas
- Cache warming: Pre-load frequently accessed product pages into Redis
- Queue system: Use SQS/RabbitMQ for order processing to avoid DB bottlenecks

2.4 Load Balancing Configuration

Use a Round Robin or Least Connections algorithm on the Application Load Balancer. Enable sticky sessions only for the checkout flow. Set health checks every 30 seconds to auto-remove unhealthy instances.

Expected Outcome	The architecture can handle 10x normal traffic (e.g., 500,000 concurrent users) with 99.99% uptime and sub-2-second response times during peak load.

Case Study 3

Website Migration: Shared to Cloud

Scenario	A company plans to migrate its website from shared hosting to cloud hosting for better performance, reliability, and scalability.

Activity: Migration Plan

3.1 Pre-Migration Assessment

Before migration begins, conduct a full audit of the existing environment. Document all dependencies, database schemas, file structures, email configurations, and third-party integrations.

Phase	Activity	Duration
Assessment	Inventory of files, DBs, configs	Week 1
Planning	Select cloud provider, design architecture	Week 1-2
Backup	Full backup of files and database	Week 2
Setup	Provision cloud environment	Week 2-3
Migration	Transfer data and configure DNS TTL	Week 3
Testing	Functional, performance, security tests	Week 3-4
Go-Live	DNS cutover and monitoring	Week 4

3.2 Backup Strategy

- Create full cPanel/FTP backup of all website files
- Export MySQL/PostgreSQL databases with timestamps
- Store backups in 3 locations: local, cloud storage (S3), and external drive
- Verify backup integrity by restoring to a staging environment

3.3 Downtime Minimisation

Key Technique	Reduce DNS TTL to 300 seconds (5 minutes) 48 hours before cutover. This allows DNS propagation to complete within minutes instead of 24–48 hours during the actual switch.

3.4 Testing Checklist

- Functional testing: All pages load correctly, forms submit, payments process
- Performance testing: Page load times compared against baseline (target: 30% improvement)
- Security testing: SSL certificate active, headers configured, no open ports
- SEO validation: Redirects working, canonical URLs correct, sitemap accessible
- Email validation: MX records pointing to correct mail server

3.5 Rollback Plan

If critical issues arise post-migration, immediately revert DNS to point back to the original shared host. Maintain the old shared hosting account for 30 days post-migration as a safety net.

Case Study 4

Cost Optimisation in Cloud Hosting

Scenario	A startup is experiencing unexpectedly high cloud bills despite relatively modest traffic. The team needs to identify waste and implement cost-saving measures.

Activity: Cloud Cost Optimisation Strategy

4.1 Root Cause of High Bills

Issue	Typical Overspend	Solution
Over-provisioned instances	40% waste	Right-size to actual usage
Idle resources 24/7	35% waste	Use scheduled scaling / spot instances
No lifecycle policies on storage	15% waste	Auto-archive old data to cold storage
Redundant data transfer	10% waste	Use CDN for static content

4.2 Optimisation Techniques

- Reserved Instances: Purchase 1-year Reserved Instances for predictable workloads — saves up to 40% vs on-demand pricing
- Spot/Preemptible Instances: Use for batch jobs, background tasks, and dev environments — up to 90% discount
- Auto-scaling: Scale down during off-peak hours (e.g., 2 AM–6 AM) to reduce running instances
- Storage Tiering: Move infrequently accessed data from SSD to HDD or cold storage (AWS S3 Glacier)
- Right-sizing: Analyse CloudWatch/Cloud Monitoring metrics and downgrade over-provisioned instances
- CDN for static assets: Offload images, CSS, and JS to CDN — reduces origin server bandwidth costs

4.3 Monitoring Tools

Tools	AWS Cost Explorer, GCP Billing Reports, Azure Cost Management — set up budget alerts at 80% and 100% of monthly budget threshold.

4.4 Projected Savings

Strategy	Estimated Saving
Reserved Instances	30–40%

Spot Instances for dev/test	50–70%
Auto-scaling (off-peak)	20–30%
Storage optimisation	10–20%
CDN for static assets	15–25%

Expected Outcome	A startup spending $2,000/month on cloud infrastructure can realistically reduce costs to $800–$1,200/month (40–60% reduction) by implementing these strategies systematically.

Case Study 5

Managed vs Unmanaged Hosting

Scenario	A startup with no dedicated IT team needs to choose between managed and unmanaged hosting. They have limited technical expertise but need a reliable hosting environment.

Activity: Role Play Decision Analysis

5.1 Role Play Scenario

Characters: CEO (focused on cost savings), CTO (focused on flexibility), Operations Manager (focused on reliability). Scenario: The startup is choosing between managed and unmanaged VPS hosting for their SaaS product.

5.2 Comparison Matrix

Feature	Managed Hosting	Unmanaged Hosting
Server Setup	Provider handles all	You configure everything
OS Updates	Automatic	Manual responsibility
Security Patches	Provider applies	Self-managed
Technical Support	24/7 from provider	Community forums only
Cost	Higher ($50–$300/mo)	Lower ($10–$80/mo)
Flexibility	Limited customisation	Full root access
Time Investment	Minimal (focus on product)	High (server management)
Best For	Non-technical teams	DevOps-capable teams

5.3 CEO Perspective

CEO argues for unmanaged hosting to save $200/month. However, the hidden costs include: server downtime losses, developer time diverted from product development, and potential security incidents. These easily exceed the $200 monthly saving.

5.4 CTO Perspective

CTO acknowledges the flexibility of unmanaged hosting but concedes the team of 5 engineers cannot afford 10+ hours per week on server administration. This time is better spent building features.

5.5 Decision & Justification

Decision	Managed Hosting is the clear choice for this startup. The productivity cost of unmanaged hosting (engineering hours on server maintenance) far exceeds the monthly premium. As the company grows and hires a DevOps engineer, a migration to unmanaged or hybrid hosting can be reconsidered.

- Short term (0–2 years): Managed cloud hosting (e.g., Heroku, Render, AWS Elastic Beanstalk)
- Medium term (2–4 years): Transition to managed Kubernetes (GKE, EKS)
- Long term (4+ years): Consider unmanaged infrastructure with dedicated DevOps team

Case Study 6

Cloud Hosting Architecture Design

Scenario	A company wants to launch a scalable global web application capable of serving users across multiple continents with low latency and high availability.

Activity: Architecture Design & Explanation

6.1 Architecture Components

Component	Service Example	Function
DNS	Route 53 / Cloud DNS	Latency-based routing to nearest region
CDN	CloudFront / Fastly	Edge caching for static content
Load Balancer	ALB / GCP LB	Distribute traffic across app servers
Web Servers	EC2 Auto Scaling Group	Handle HTTP requests (stateless)
App Layer	ECS / GKE Containers	Business logic processing
Cache Layer	ElastiCache (Redis)	Session and query caching
Primary Database	RDS Multi-AZ (PostgreSQL)	ACID-compliant transactional data
Read Replicas	RDS Read Replicas	Offload read-heavy queries
Object Storage	S3 / GCS	Static files, images, backups
Message Queue	SQS / Pub/Sub	Async task processing
Monitoring	CloudWatch / Datadog	Metrics, logs, alerts

6.2 Multi-Region Strategy

Deploy the application in at least 3 geographic regions (e.g., US East, EU West, AP Southeast). Use active-active configuration for the application layer and active-passive for databases with cross-region replication.

6.3 Data Flow Description

- User request hits the CDN edge node — static content served immediately from cache
- Dynamic requests forwarded via CDN to the regional Load Balancer
- Load Balancer distributes to available App Server instances using Least Connections algorithm
- App Server checks Redis cache — cache hit returns response in <10ms
- Cache miss: App Server queries Primary DB or nearest Read Replica
- Asynchronous tasks (emails, reports) pushed to SQS queue for background processing

- All logs streamed to centralised monitoring for real-time alerting

SLA Target	This architecture achieves 99.99% uptime (< 52 minutes downtime per year) through redundancy at every tier, across multiple availability zones and regions.

Case Study 7

CDN and Website Performance

Scenario	A website with predominantly local users is experiencing very slow load times for international visitors. Page load times exceed 8 seconds for users in Europe and North America.

Activity: CDN-Based Performance Solution

7.1 Root Cause Analysis

Without a CDN, every user request travels from their browser to the origin server (wherever it is physically located) and back. For a user in Germany accessing a server in Mumbai, this round trip adds 200–400ms of latency per request — and a typical webpage makes 50–100 requests.

User Location	Without CDN	With CDN	Improvement
Local (same city)	120ms	80ms	33% faster
Same country	200ms	90ms	55% faster
Same continent	400ms	100ms	75% faster
Different continent	800ms+	110ms	86% faster

7.2 CDN Implementation Plan

- Select CDN provider: Cloudflare (free tier available), AWS CloudFront, or Fastly
- Configure DNS: Point domain's CNAME to CDN edge network
- Define caching rules: Cache HTML for 60 seconds, CSS/JS for 30 days, images for 1 year
- Enable compression: Brotli/GZIP compression for all text-based assets
- Activate HTTP/2 and HTTP/3 (QUIC) for multiplexed connections
- Configure cache purging: Automate purge on content deployment

7.3 Additional Optimisations

- Enable image optimisation: WebP conversion and responsive images via CDN
- Minify assets: CSS, JavaScript, and HTML minification at the CDN edge
- Preload critical resources: Use <link rel=preload> for fonts and hero images
- Lazy loading: Defer loading of below-fold images and videos

Expected Results	Implementation of a CDN combined with image optimisation and compression typically reduces page load times by 60–80% for international users and reduces origin server bandwidth usage by 70%, lowering hosting costs.

Case Study 8

Security in Web Hosting

Scenario	A website suffers a data breach exposing customer PII (names, emails, credit card data) due to weak security configurations. The company faces regulatory fines and reputational damage.

Activity: Security Audit & Improvement Plan

8.1 Breach Investigation Findings

Vulnerability	Severity	Impact
Outdated CMS (WordPress 4.9)	Critical	Known remote code execution exploits
Default admin credentials	Critical	Attacker gained admin panel access
No Web Application Firewall	High	SQL injection not blocked
Unencrypted database connections	High	Data interceptable in transit
No rate limiting on login	High	Brute force attacks succeeded
Misconfigured S3 bucket (public)	High	Customer files publicly accessible
No 2FA on admin accounts	Medium	Single point of compromise

8.2 Immediate Remediation Steps

- Change all admin passwords immediately and revoke all active sessions
- Update CMS, plugins, and themes to latest versions
- Enable Web Application Firewall (Cloudflare WAF or AWS WAF)
- Implement 2FA on all administrative accounts
- Review and restrict all S3 bucket permissions — make private, use signed URLs
- Enable SSL/TLS for all database connections (require encrypted connections)

8.3 Long-Term Security Framework

Security Domain	Controls to Implement
Access Control	Principle of Least Privilege, RBAC, MFA everywhere
Network Security	VPC with private subnets, security groups, NACLs

Application Security	Input validation, parameterised queries, OWASP Top 10 review
Data Protection	Encryption at rest (AES-256), in transit (TLS 1.3)
Monitoring	SIEM, intrusion detection, centralised logging with alerts
Compliance	GDPR data processing audit, PCI-DSS for payment handling
Incident Response	Documented IR plan, regular drills, breach notification procedure

Compliance Note	Under GDPR, a data breach must be reported to the supervisory authority within 72 hours of discovery. Failure to do so can result in fines of up to 4% of annual global turnover.

Case Study 9

SaaS Application Hosting — Online Exam System

Scenario	A team develops an online exam system that will serve thousands of simultaneous users from universities and schools. The system must be highly available during examination periods.

Activity: SaaS Hosting Model Design

9.1 SaaS Architecture Requirements

- Multi-tenancy: Single platform serving multiple institutions with data isolation
- High availability: 99.9% uptime guaranteed, especially during exam windows
- Scalability: Handle sudden spikes when 10,000 students start an exam simultaneously
- Security: Exam integrity, anti-cheating measures, encrypted data storage
- Disaster recovery: RPO < 1 hour, RTO < 4 hours

9.2 Multi-Tenant Architecture

Layer	Design Choice	Reason
Tenant Isolation	Schema-per-tenant (PostgreSQL)	Strong isolation with shared infrastructure
Authentication	OAuth 2.0 + JWT with tenant claims	Secure, stateless, scalable
Compute	Kubernetes with namespace isolation	Resource quotas per tenant
Data	Encrypted at rest, tenant-keyed	GDPR compliance
Monitoring	Tenant-specific dashboards	Performance visibility per institution

9.3 Exam Day Scaling Strategy

- Pre-schedule scale-out: 30 minutes before exam start, auto-scale to 3x baseline capacity
- Queue-based submission: Exam answer submissions go through SQS queue to prevent DB overload
- Read replicas: 5 read replicas to handle question fetching (read-heavy workload)
- Sticky sessions disabled for question serving (stateless); enabled only for active exam sessions
- Real-time monitoring: PagerDuty alerts if response time exceeds 2 seconds

9.4 Data & Compliance

Data Residency	Store institutional data in compliance with local regulations. Indian universities: host on AWS Mumbai (ap-south-1). EU institutions: AWS Frankfurt (eu-central-1). Implement data residency controls at the tenant onboarding level.

9.5 SaaS Pricing Model

Tier	Users	Features	Price
Starter	Up to 500	Basic exam, MCQ	$99/month
Professional	Up to 5,000	+ Proctoring, analytics	$499/month
Enterprise	Unlimited	+ SSO, custom domain, SLA	Custom pricing

Case Study 10

Website Failure — Downtime Root Cause Analysis

Scenario	A major e-learning platform experiences a global outage lasting 6 hours. Investigation reveals it was caused by a DNS misconfiguration during a routine maintenance update.

Activity: Root Cause Analysis & Preventive Strategies

10.1 Incident Timeline

Time	Event
14:00 UTC	Engineer updates DNS records for new CDN integration
14:05 UTC	Incorrect TTL value (0s) set on A records — causes resolver storm
14:08 UTC	DNS resolver flood overwhelms authoritative name servers
14:12 UTC	Website becomes unreachable globally — monitoring alerts trigger
14:20 UTC	Incident response team assembled
15:30 UTC	Root cause identified — DNS misconfiguration
16:00 UTC	Fix applied — correct TTL values restored
20:00 UTC	Full DNS propagation complete — service restored globally

10.2 Root Cause Analysis (5 Whys)

- Why did the website go down? DNS records were unreachable due to name server overload.
- Why were name servers overloaded? TTL was set to 0, causing every DNS query to hit the authoritative server instead of using cached responses.
- Why was TTL set to 0? The engineer misunderstood the CDN configuration requirement.
- Why was there a misunderstanding? There was no mandatory review process for DNS changes.
- Why was there no review process? Change management procedures for DNS were not defined in the runbook.

10.3 Preventive Measures

Key Finding	The root cause was a process failure, not a technical one. The fix requires both technical safeguards and process improvements.

- Implement change management: All DNS changes require peer review and approval before deployment
- Staging environment: Test DNS configurations in a staging zone before production
- DNS monitoring: Continuous monitoring of DNS resolution times with automated alerts
- Minimum TTL policy: Set organisation-wide minimum TTL of 300 seconds
- Runbook documentation: Detailed step-by-step procedures for all infrastructure changes
- Chaos engineering: Regular planned failure drills to test recovery procedures
- Secondary DNS: Use multiple DNS providers (primary + secondary) for redundancy

Case Study 11

Enterprise / Government Cloud Adoption

Scenario	A state government department managing citizen services (tax filing, license renewal, welfare applications) plans to migrate from on-premise data centres to cloud infrastructure.

Activity: Debate — Advantages vs Challenges

11.1 Arguments For Cloud Adoption

Advantage	Detail
Cost Reduction	Eliminates capital expenditure on hardware; OPEX model aligns spending with usage
Scalability	Handle peak loads (tax season, elections) without over-provisioning year-round
Disaster Recovery	Geo-redundant backups ensure continuity even during physical disasters
Innovation Speed	Access to AI, ML, and big data services accelerates citizen service improvements
Remote Access	Government staff can work from anywhere — critical post-pandemic
Compliance	Major cloud providers are certified for government workloads (FedRAMP, ISO 27001)

11.2 Arguments Against / Challenges

Challenge	Detail
Data Sovereignty	Citizen data must remain within national borders — requires local data centres
Vendor Lock-in	Dependence on a single cloud provider creates future negotiation risks
Security Concerns	Perceived risk of sensitive data being exposed to foreign entities
Legacy Systems	Many government systems run on COBOL/mainframes — migration is complex
Staff Skills Gap	Government IT workforce may lack cloud expertise — training required
Internet Dependency	Cloud services require stable internet — rural areas may face access issues

11.3 Recommended Approach: Government Cloud Strategy

- Adopt a Hybrid Cloud model: Sensitive citizen data stays on-premise or in a sovereign cloud; non-sensitive services migrate to public cloud
- Establish a Government Cloud Marketplace: Negotiate with multiple approved vendors to avoid lock-in
- Data classification policy: Define which data categories can reside in public vs private cloud
- Phased migration: Begin with non-critical services (websites, email) before migrating core systems
- Staff training: Mandatory cloud certification programme for IT staff

Reference	India's National Cloud (MeghRaj) initiative and the UK Government Cloud (G-Cloud) framework provide models for sovereign cloud adoption that balance innovation with data sovereignty requirements.

Case Study 12

Local Business Website Hosting

Scenario	A small local bakery wants to establish an online presence with a website showcasing their menu, location, and online order form. Budget is very limited (< $20/month).

Activity: Hosting Recommendation with Justification

12.1 Business Requirements Analysis

Requirement	Detail
Expected Traffic	500–2,000 visitors/month (local audience)
Content Type	Static pages + simple PHP form
Budget	< $20/month total (hosting + domain)
Technical Expertise	None — owner manages the website
Features Needed	Email hosting, SSL certificate, file manager
Growth Expectation	Stable — unlikely to exceed 5,000 visitors/month

12.2 Hosting Options Evaluation

Provider Type	Monthly Cost	Suitability	Verdict
Shared Hosting (Hostinger)	$2–4/month	Ideal for static/low-traffic	RECOMMENDED
Free Hosting (000webhost)	$0/month	Unreliable, ads shown	NOT recommended
WordPress.com	$4–9/month	Easy but limited customisation	Acceptable
VPS	$20–40/month	Overkill for this use case	Not suitable
Cloud Hosting	$30+/month	Overkill + complex management	Not suitable

12.3 Recommended Solution

Recommendation	Shared Hosting on Hostinger or Bluehost Starter Plan. At $2–4/month, it includes free SSL, email accounts, a drag-and-drop website builder, and 99.9% uptime SLA — perfectly matched to a local bakery's needs and budget.

12.4 Complete Setup Plan

- Domain registration: Purchase .in or .com domain for ~$10–12/year
- Hosting plan: Hostinger Single or Premium plan — includes free domain and SSL
- Website builder: Use included builder (no coding required) to create menu and contact pages
- Google Business Profile: List the bakery for free to appear in local Google Maps results
- Email setup: Configure name@bakerydomain.com using cPanel email manager
- Analytics: Install Google Analytics 4 (free) to track visitor behaviour

12.5 Total Monthly Cost Breakdown

Item	Annual Cost	Monthly Equivalent
Hostinger Premium Hosting	$~28/year	$2.32/month
Domain (.com)	$~12/year	$1.00/month
SSL Certificate	$0 (included)	$0/month
Email Hosting	$0 (included)	$0/month
Total	$~40/year	$3.32/month

Outcome	The bakery gets a professional online presence with business email, SSL security, and a reliable hosting environment for under $4/month — well within the stated budget while providing all required features.

Summary & Key Takeaways

These twelve case studies collectively illustrate the breadth and depth of decisions involved in web hosting and cloud technology management. From a local bakery choosing shared hosting to government departments navigating cloud sovereignty, each scenario demonstrates that the 'right' hosting solution is always context-dependent.

Core Principles

- Match the hosting solution to actual business requirements — over-engineering wastes money; under-engineering risks failure
- Security is non-negotiable regardless of budget or scale — breaches cost far more than prevention
- Scalability must be planned in advance — reactive architecture changes during crises are costly and risky
- Cost optimisation is an ongoing process, not a one-time configuration
- Human factors (training, process, documentation) are as important as technical controls
- Cloud adoption is a journey — hybrid approaches often provide the best balance of control and agility

Case Study	Key Technology	Primary Lesson
1. Hosting Types	Shared / VPS / Cloud	Right-size to traffic and budget
2. Peak Traffic	Auto-scaling, Load Balancing	Design for maximum expected load
3. Migration	DNS, Backup, Testing	Plan thoroughly to minimise downtime
4. Cost Optimisation	Reserved Instances, CDN	Continuous monitoring prevents bill shock
5. Managed vs Unmanaged	Managed Cloud Services	Non-technical teams need managed solutions
6. Architecture Design	CDN, LB, Multi-AZ DB	Redundancy at every tier ensures uptime
7. CDN Performance	Edge Caching, Compression	CDN solves global latency problems
8. Security	WAF, Encryption, IAM	Defence-in-depth protects against breaches
9. SaaS Hosting	Multi-tenancy, K8s	Isolation and scalability are SaaS foundations
10. Downtime RCA	DNS, Change Management	Process failures cause most outages
11. Government Cloud	Hybrid Cloud, Sovereignty	Public sector needs special compliance

12. Small Business	Shared Hosting	Simple solutions suit simple needs